

Contents

Chapter 1	Maths for Quantum Mechanics	Page 2
1.1	Why linear algebra? Classical Physics — 2 • Quantum — 2	2
1.2	Kets and Wavefunctions Infinite sets — 3	3
1.3	Hilbert Space	4
1.4	Inner products Dot product — 5 • Definition — 6 • Finding Coefficients — 7 • Calculating the inner product — 7	5
1.5	Dirac deltas and Wavefunction inner products	8
1.6	Bra-ket notation	8
1.7	Observables as operators	8
1.8	Calculating Probabilities	8
1.9	Hermitian operators	8
1.10	Commutator and the Uncertainty principle	8
1.11	Unitary operators	8
1.12	Classical Generators	8
1.13	Shrodinger equation	8
1.14	Momentum operator	8
Chapter 2	Multiple Systems	Page 9
2.1	Classical states	9
2.2	Probabilistic states	10
Chapter 3	Quantum Computing	Page 11
3.1	Qbits	11
3.2	Multi-Qbit Systems	11
3.3	Quantum Gates	11
3.4	Quantum Circuits	12
3.5	Circuit Optimization	12
Chapter 4	Hoare Logic and Automated Verification	Page 13
4.1	Hoare Logic	13
4.2	Weakest Precondition Calculus	13
4.3	SMT Solvers	14

Chapter 1

Maths for Quantum Mechanics

1.1 Why linear algebra?

1.1.1 Classical Physics

Physical quantities:

- **Single-valued**
- **Continuous**

So they can be mapped as a **continuous function**.

1.1.2 Quantum

In the quantum world, physical values can also be:

- **Probabilistic**
- **Discrete**

So we can't use a continuous function.

Probabilisticness

We can represent each possible outcome of our value with a mathematical object M_i , and we need to combine these giving each a specific probability.

$$a_1 M_1 \cdot a_2 M_2 \cdot \dots \cdot a_n M_n$$

where a_i encodes the likelihood of its associated value to occur, and $\cdot$ is an operation that combines the values. Informally, we can see how this structure is similar to a **linear equation**, we'll come back to that.

Discreteness

What mathematical object can we use to extract discrete values? Following our linear combination lead, we can think of representing physical quantities as **matrices** (linear operators), from which we can think to extract the possible discrete values.

1.2 Kets and Wavefunctions

Definition 1.2.1: Vector Space

A set of objects (called vectors) that satisfy the following conditions $\forall u, v, w$ vectors and $\forall a, b \in \mathbb{C}$:

- $u + v$ is a vector
- au is a vector
- $-u$ is a vector
- 0 vector exists
- $1u = u$
- $u + (v + w) = (u + v) + w$
- $u + v = v + u$
- $a(bu) = (ab)u$
- $a(u + v) = au + av$
- $(a + b)u = au + bu$

Note:

This definition doesn't mention arrows or lists of numbers or arrows, we can use any type of object that follows the structure.

Let's try to represent our particle as a vector in a vector space, a **quantum state** ψ :

Definition 1.2.2: Quantum State

Vector that holds all the physical properties of a particle, which should all be extractable at any moment in time, as well as all the associated probabilities. Usually represented in bra-ket notation as:

$$|\psi\rangle$$

We want our state to represent a linear combination of all possible outcomes of a measurement:

$$|\psi\rangle = a_1 |E_1\rangle + \dots + a_n |E_n\rangle$$

Where $E_1, \dots, E_n$ are all the possible values of the particle's energy, and the coefficients have something to do with the probability of the outcome. This is called a **superposition**.

But the state should hold values for **all** physical properties, so—looking at angular momentum as an example—we must also have that:

$$|\psi\rangle = b_1 |L_1\rangle + \dots + b_n |L_n\rangle$$

So the same state also encodes a different linear combination.

1.2.1 Infinite sets

It's possible for the set of discrete values of a property to be infinite. Take for example the energy: we can theoretically keep adding more and more without a limit, making the set of all possible values still discrete, but infinite:

$$|\psi\rangle = \sum_{i=1}^{\infty} a_i E_i$$

Note:

There are still some problems that can arise by allowing infinity without proper constraints, as we'll explore in the next chapter.

Furthermore, some properties like position, have continuous values. These can't be represented with a normal linear sum.

Let's say that a particle's position can be in any point on the continuous line between a and b . In this case we need a continuous sum, which can be performed with an integral:

$$|\psi\rangle = \int_a^b c(x)|x\rangle dx$$

Note that instead of a list of coefficients for each value we now have a function $c(x)$ that encodes probability for each continuous value. This function is called a **wavefunction**.

1.3 Hilbert Space

So the state of our particle can be expressed as a vector in a vector space, and this vector can be in an infinite (even continuous) linear combination of outcomes. We will later prove that this list of outcomes actually forms a basis for the vector space.

So, seen as the dimension of a vector space is equal to the size of the basis, our quantum state has to live in an *infinite-dimensional space*.

Example 1.3.1 (Polinomials)

An example for an infinite vector space is the polinomials over the real numbers. It's straight forward to check that polinomials fit the definition of a vector space. A basis for such a space can be

$$\{x^0, x^1, x^2, x^3, \dots\}$$

seen as all polinomials are by definition a linear combination of these powers.

Now let's see what happens when we let our space become infinite-dimensional.

$$p_{\text{vector}}(x) = 1x^0 + 1x^1 + \frac{1}{2}x^2 + \frac{1}{6}x^3 + \frac{1}{24}x^4 + \frac{1}{120}x^5 + \dots$$

As we all remember from our lessons with Morbidelli, this is the Taylor serier for e^x :

$$e^x = \lim_{n \rightarrow +\infty} \sum_{i=0}^n \frac{1}{i!} x^i$$

But e^x is definetly not a polinomial, so what happened? How did we leave our vector space?

The core problem that causes the unexpected behaviour in the example is that infinity is a *concept*, not a number. This means that when we let the number of terms approach the idea of infinity, we actually end just outside our set of valid elements.



This is the reason why introducing infinity to a vector space is risky. Going back to our quantum states, we need infinite dimensions but we also want to avoid leaving our vector space. How can we solve this? Easy, we just add a rule:

$$\sum_i^\infty |E_i\rangle \rightarrow |\psi\rangle$$

meaning all convergent sums of vectors must converge to an element inside the vector space. This is basically the definition of a **Hilbert space**:

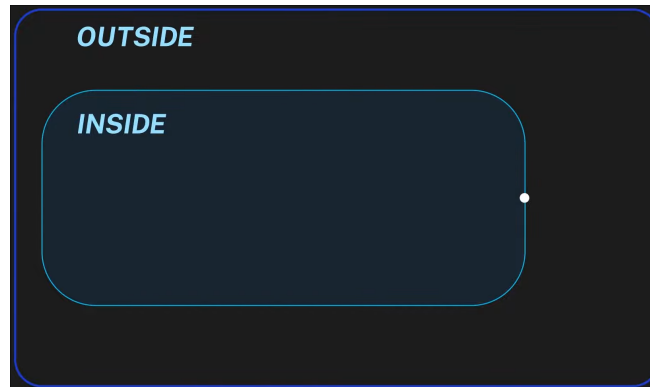
Definition 1.3.1: Hilbert Space

A vector space equipped with an **inner product** that is **Cauchy complete**.

Let's break this down:

- An inner product is just a generalized dot product (we'll explore this later)
- Cauchy complete means that every convergent sequence of vectors converges to an element inside the vector space.

So basically we can say that a vector space is "complete" when it includes its "edges"



1.4 Inner products

In the definition of a Hilbert space, we mentioned the *inner product* and we described it as a generalization of the dot product.

Let's first understand what the dot product is used for.

1.4.1 Dot product

Why is the dot product useful for vector spaces:

- It allows us to define a notion of **angle** and **orthogonality**:

$$\vec{v} \cdot \vec{w} = |\vec{v}| |\vec{w}| \cos \theta$$

and

$$\vec{v} \perp \vec{w} \iff \vec{v} \cdot \vec{w} = 0$$

- It helps us define the **length** of an abstract vector:

$$|\vec{v}| = \sqrt{\vec{v} \cdot \vec{v}}$$

We would like to use these properties in our quantum space, so we need to find a way to abstract the dot product for it to work on vectors other than lines and number columns.

1.4.2 Definition

We want a function that takes in two vectors and outputs a number (that can be complex)

$$\text{InProd}(|\psi\rangle, |\phi\rangle) = c$$

This notation isn't very clean, so in bra-ket notation we use

$$\langle\psi|\phi\rangle$$

It should also be linear, but for now let's only consider the right side

$$\begin{aligned}\langle\psi|\phi + \zeta\rangle &= \langle\psi|\phi\rangle + \langle\psi|\zeta\rangle \\ \langle\psi|a\phi\rangle &= a\langle\psi|\phi\rangle\end{aligned}$$

We would assume that the inner product should also be commutative, allowing us to "flip" the operands and thus also being linear on the left side. But sadly a problem arises, as we'll now prove:

Proof: Let's assume that inner product is commutative and prove it's absurd. Consider a vector ϕ such that

$$\langle\phi|\phi\rangle = 1$$

Let's now look at the inner product

$$\langle i\phi|i\phi\rangle$$

By using right side linearity and the commutative property, we get

$$-1\langle\phi|\phi\rangle = -1$$

But this means that the length of the vector $i\phi$ is $\sqrt{-1} = i$, which is impossible because a length has to be a real number. $\textcircled{Q}$

To fix this issue, we need to add a condition: each time we "flip" the vectors of an inner product, we must add a complex conjugate

Example 1.4.1 (Correcting the previous case)

$$\langle i\phi|i\phi\rangle = i\langle\phi|\phi\rangle = i\langle\phi|i\phi\rangle^* = ii^*\langle\phi|\phi\rangle^* = ii^*\langle\phi|\phi\rangle = ii^* = 1$$

Lastly, to ensure that the magnitude of a vector makes sense, we need to state this rather obvious rule

$$|\psi| = \langle\psi|\psi\rangle \neq 0 \iff |\psi\rangle \neq 0$$

We can now finally define the inner product.

Definition 1.4.1: Inner Product

An inner product $\langle\psi|\phi\rangle$ is a map from vectors to scalars that satisfies the following:

- Right-side linear
- Complex conjugate commutation
- Only zero vector has length 0

So basically just the dot product with extra rules to make sure we can still define length. With these rules, we can also derive that the inner product is *antilinear* on the left

$$\langle a\psi + b\zeta|\phi\rangle = a^*\langle\psi|\phi\rangle + b^*\langle\zeta|\phi\rangle$$

Like with the dot product, we say that

- The **magnitude** of a vector ψ is $\sqrt{\langle\psi|\psi\rangle}$
- If $\langle\psi|\phi\rangle = 0$ then the two vectors are **orthogonal**

1.4.3 Finding Coefficients

Let's say we have an **orthonormal** basis $\{|E_i\rangle\}$

$$\langle E_i | E_j \rangle = \delta_{ij}$$

where δ_{ij} is the *Kronecker* delta.

We would now like to know how to get a particular coefficient c_k of a state $|\psi\rangle = \sum_i c_i |E_i\rangle$. Well, we can use the inner product

$$\begin{aligned}\langle E_k | \psi \rangle &= \langle E_k | \left(\sum_i c_i |E_i\rangle \right) \\ &= \sum_i c_i \langle E_k | E_i \rangle \\ &= \sum_i c_i \delta_{ki} = c_k\end{aligned}$$

So, if we have an orthonormal base, to get a coefficient we just need to calculate the inner product between the associated basis vector and the state vector.

1.4.4 Calculating the inner product

Given two states $|\psi\rangle = \sum_i |a_i E_i\rangle, |\phi\rangle = \sum_j |b_j E_j\rangle$ where $|E_i\rangle$ is an orthonormal basis of the vector space, we can calculate the value of the inner product

$$\begin{aligned}\langle \psi | \phi \rangle &= \left(\sum_i \langle a_i E_i | \right) \left(\sum_j |b_j E_j\rangle \right) \\ &= \sum_j b_j \left(\sum_i \langle a_i E_i | \right) |E_j\rangle \\ &= \sum_i \sum_j a_i^* b_j \langle E_i | E_j \rangle \\ &= \sum_i a_i^* b_i\end{aligned}$$

If we assume that all coefficients are real, we get the definition of the dot product

$$\langle \psi | \phi \rangle = \sum_i a_i b_i$$

This is very cool because we made no mention of the dot product when defining the inner product, we just copied some of its properties.

- 1.5 Dirac deltas and Wavefunction inner products
- 1.6 Bra-ket notation
- 1.7 Observables as operators
- 1.8 Calculating Probabilities
- 1.9 Hermitian operators
- 1.10 Commutator and the Uncertainty principle
- 1.11 Unitary operators
- 1.12 Classical Generators
- 1.13 Shrodinger equation
- 1.14 Momentum operator

Chapter 2

Multiple Systems

2.1 Classical states

Suppose we have just two systems:

- X is a system with classical state set Σ
- Y is a system with classical state set Γ

we can place them side-by-side to obtain a single classical system.

Definition 2.1.1: Compound classical system (pair)

The system created by joining two classical states X, Y :

$$XY$$

Question 1

What are the classical states of (X, Y) ?

The classical state set of (X, Y) is the *Cartesian product* of the two sets:

$$\Sigma \times \Gamma = \{(a, b) : a \in \Sigma \text{ and } b \in \Gamma\}$$

Note:

We're making the assumption that we know which system is which, and there's no confusion between them.

This generalizes well for n systems:

Definition 2.1.2: Compound classical system

Given n classical systems $X_1, \dots, X_n$, the **compound system** formed by them is written as an n -tuple:

$$(X_1, \dots, X_n)$$

or as a string:

$$X_1 \dots X_n$$

Proposition 2.1.1

The classical state set of $X_1 \dots X_n$ with states $\Sigma_1, \dots, \Sigma_n$ is the set:

$$\Sigma_1 \times \dots \times \Sigma_n$$

Note:

As per convention, cartesian products of states are ordered *lexicographically*, and significance decreases from left to right.

2.2 Probabilistic states

A probability is associated to each Cartesian product of the classical state sets of the individual systems.

Definition 2.2.1: Statistical independence

Given a probabilistic state of (X, Y) , we say that X and Y are independent if, $\forall a \in \Sigma, \forall b \in \Gamma$:

$$Pr((X, Y) = (a, b)) = Pr(X = a)Pr(Y = b)$$

By expressing the state of (X, Y) as a vector:

$$|\pi\rangle = \sum_{(a,b) \in \Sigma \times \Gamma} p_{ab} |ab\rangle$$

Chapter 3

Quantum Computing

Whereas classical computers are based on bits that can either be switched "on" or "off", quantum computers work by carefully modifying the state of a quantum system. This fundamental difference in models of computation requires a complete remodelling of the mathematical representation that describes how a computer behaves.

3.1 Qbits

Qbits, short for quantum bits, are the fundamental element of quantum information. They are implemented as a binary property of a quantum system. This means that when said property is measured, the output is either "on" or "off", just like a classical bit. The difference is that quantum superposition allows the system to be in multiple states at the same time, and when the property is observed it stochastically collapses into a single base state. To model this crazy behaviour, the state of a qbit $|\psi\rangle$ can be represented as a linear combination:

$$|\psi\rangle = c_0|0\rangle + c_1|1\rangle$$

where $|0\rangle$ and $|1\rangle$ indicate the "off" and "on" states and $c_0, c_1 \in \mathbb{C}$ such that $|c_0|^2$ and $|c_1|^2$ are the probabilities of the qbit being in the relative state when measured. The reason for using the squared modulus of complex numbers as opposed to directly using probabilities as coefficients is that this allows for addition to lower the resulting probabilities. This property is needed to correctly model quantum interference,

So we can represent a qbit as a normalised vector in $\mathbb{C}^2$ with $\begin{pmatrix} 1 \\ 0 \end{pmatrix} = |0\rangle$ and $\begin{pmatrix} 0 \\ 1 \end{pmatrix} = |1\rangle$ as base:

$$|\psi\rangle = \begin{pmatrix} c_0 \\ c_1 \end{pmatrix}$$

3.2 Multi-Qbit Systems

When adding a qbit to a quantum system, the number of possible base states doubles, meaning that an n -qbit system's state is represented as a vector in $\mathbb{C}^{2^n}$. Given two quantum systems, their composition is represented by the tensor product of their vector spaces. (give example and code)

The state of a multi-qbit system can't be described as the product of single-qbit states, as shown by the following example:

(bell pair example)

This mathematical fact describes a unique quantum behaviour known as entanglement, which is when two

3.3 Quantum Gates

Because we're representing a quantum state as a vector, any operator that acts on qbits will be an endomorphism, and thus a linear application with an associated square matrix $U \in M(\mathbb{C})n \times n$.

As described in Shrodinger's equation (cite maybe?), a closed quantum system can only undergo unitary evolution, thus the only way we can we can manipulate our quantum state without it collapsing is through unitary matrices. A matrix is unitary if (define unitary matrix and how they're isometric)...

So all possible operators that act on a closed quantum system, known as quantum gates, are isomorphic to the group of unitary matrices of size n . As a result, quantum gates are limited to only reversible operations that cannot clone a qbit's state to another without collapsing the original.

A set of gates is universal if, through composition, they can simulate any other quantum gate. The set of gates that will be used in this thesis is the following: (TODO: fix a gate set)

3.4 Quantum Circuits

A quantum program can be represented as a single unitary matrix acting on all qbits, depicted as follows:

(image)

where the wires represent the life span of the qbits. The single complex gate can be broken down into a composition of multiple simpler gates that are applied to a subset of qbits, so we can depict how the program builds the corresponding circuit sequentially:

(image)

We can define some structural properties of such a circuit:

- Gate count:
- Width:
- Depth:

3.5 Circuit Optimization

It's possible that the construction of the circuit determined by the quantum program isn't the most efficient decomposition, given a fixed gate set, of the ?? program matrix ?? with respect to some of the circuit's properties. This means that the circuit can be optimized. There are different ways a quantum circuit can be transformed into an equivalent, more optimized, one.

(describe different optimization techniques, ending with triviality conditions (the one ill use))

Chapter 4

Hoare Logic and Automated Verification

Formally verifying a computer program means proving its correctness, with respect to a formal specification of the programming language used, by using formal or mathematical methods. This is the strongest possible way to determine if a certain block of code works as expected or not, and because of this formal verification is used mostly in security-related applications. For the purposes of this thesis, we are interested in such techniques as they don't require running or simulating quantum code in order to reason about its properties.

4.1 Hoare Logic

Hoare Logic is an axiomatic system for formally specifying what a program does. In 1969 C.A.R. Hoare introduced the Hoare Triple, written as follows:

$$\{P\}C\{Q\}$$

where:

- C is a program (or any block or single command)
- P and Q are logical assertions, called conditions, describing the program's state immediately before/after the execution of C . P is known as the precondition and Q as the postcondition.

The conditions can be written using any well-defined formal logic that allows for the required descriptiveness, which in our case is propositional logic (TODO: or first order?).

We will only be interested in the *partial correctness* of our program: if the code in C loops indefinitely, then the triple is considered vacuously true. *Total correctness* is when, given P , C is guaranteed to terminate and be in state Q .

Hoare Logic constructs proofs by using axioms and inference rules that match with the syntactic elements of a programming language. Some of the core rules are as follows:

- **Rule of Composition:** allows sequential operations to be split into separate triples
- **Assignment Axiom:** ensures that the postcondition Q holds after a variable assignment
- **If-Else Rule:**
- **While Rule:**

While Hoare Logic provides a framework for verifying proofs, given a complete set of rules and axioms, it isn't well suited for proof automation. The following section will discuss how we can do just that.

4.2 Weakest Precondition Calculus

In order to build a verification engine based on Hoare's framework, Edgar Dijkstra transformed the logic into a backward-directed deterministic function. Given a piece of code C and a postcondition Q ,

4.3 SMT Solvers